

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université Abderrahmane Mira de Bouira
Faculté des Sciences Economiques, de Gestion et Commerciales
Département des Sciences Commerciales



*Habilitation à Diriger des Recherches
(HDR)*
Polycopié de Cours du Module Informatique II (INF 2)

Initiation à l'Algorithmique

Présenté par

Dr. Zair BOUZIDI

Objectifs de l'enseignement :

Familiarisation avec de Nouveaux Concepts d'Algorithmique, de
Programmation et de l'Outil informatique.



Année 2022 – 2023

Dédicaces

À la mémoire de mon cher père Mohand Akli,

À la mémoire de ma chère mère Fatima,

À ma chère femme Melaz,

*À mon adorable fils Mohand Amokrane dit Hanin, sa femme Sabrina et leur mignon
petit garçon Dylan,*

*À mon adorable fille Sylia, son mari Djouad Ali et leur mignonne petite fille
Kawther,*

À mon adorable fille Amel et son fiancé Oughlis Mohammed Amine,

À mon adorable fille benjamine Léticia et son fiancé Makhloufi Sofiane,

À mon adorable benjamin fils Rayan,

À mes frères et sœurs et leurs familles (leurs enfants et petits-enfants),

À tous ceux que j'aime et à tous ceux qui m'aiment,

Je dédie ce modeste travail

Remerciements

Mes remerciements vont tout d'abord à mes chers collègues Dr Abdelmalek Boudries de l'Université de Béjaia et Pr Mourad Amad de l'Université de Bouira et actuellement Doyen de la Faculté des Sciences et des Sciences Appliquées de Bouira. d'avoir su me retrouver et me proposer de me sortir de l'impasse où je végétais, en m'apportant soutien constant, disponibilité, écoute, confiance, connaissance, critique, conseil précieux qui m'ont guidé tout au long de ma thèse de Doctorat qu'ils ont ensemble co-dirigés, et bien informés à la hauteur de leurs compétences et de leurs réelles qualités humaines, que je ne remercierai jamais assez.

Enfin, je tiens à remercier mes collègues de l'Université de Béjaia, notamment ceux du Département des Sciences Commerciales, qui, en plus d'être toujours disponible, j'apprends toujours avec eux et avec leurs excellents conseils.

Merci à mes parents pour leur patience et leur soutien indéfectible qui m'ont été plus qu'essentiels tout au long de mes études. Malheureusement, ils ne sont plus là pour voir le fruit de leurs efforts.

Un merci particulier à mon épouse Melaz qui est toujours à mes côtés durant toutes ces années de mariage, malgré tous les écueils de la vie.

Même si c'est un travail personnel, je tiens ici à rendre hommage et exprimer ma profonde gratitude à tous ceux qui, de près ou de loin, ont contribué à sa réalisation.

Table de Matières

Introduction	1
1 Généralités et Notions de Base	1
1.1 AVANT-PROPOS	2
1.2 Notion d'algorithme	4
1.2.1 Algorithme	4
1.2.2 Etapes de Résolution de Problèmes	4
1.2.3 Exemple	5
1.2.4 Exemple	6
1.2.5 Instance	6
1.2.6 Exemple de Résolution de Problème	6
1.2.7 Algorithme Correct	7
1.2.8 Exemple	7
1.2.9 Structure de Données	9
1.2.10 Applications	9
1.2.11 Efficacité d'un Algorithme	9
1.3 Notion de complexité	9
1.3.1 Complexité d'un Algorithme	9
1.3.2 Langage Algorithmique	10
1.4 Codage	11
1.4.1 Identifiant	11
1.4.2 Type Abstrait	11
1.4.3 Variable	12
1.4.4 Expression	12
1.4.5 Affectation	12
1.4.6 Types de base	12
1.4.6.1 Comparaison	14
2 Structures de Contrôles	18
2.1 Structures de Contrôle	18

2.2	Bloc d'instruction	18
2.2.1	Action Conditionnelle	18
2.2.1.1	Alternative	18
2.2.1.2	Alternative simple :	18
2.2.1.3	Alternative multiple :	18
2.2.2	Action Itérative	18
3	Structures de Données	26
3.1	Tableaux	26
3.1.1	Quelques exemples d'algorithmes	27
3.1.1.1	Somme des éléments d'un tableau d'entiers	27
3.1.1.2	Recherche d'un élément	27
3.1.1.3	Recherche de l'indice du premier élément minimum . .	28
3.2	Matrices	28
3.2.1	Initialisation	29
3.2.2	Somme de deux matrices réelles	29

Listes de Tables

1.1	Table Logique ET	13
1.2	Table Logique OU	13
1.3	Table Logique NON	14

Listes de Figures

1.1	Structure d'un algorithme.	5
1.2	Exemple d'algorithme.	5
1.3	Exemple d'algorithme.	7
1.4	Resolution de l'Exemple 3.	7
1.5	Resolution de l'Exemple en obtenant un Algorithme.	8
1.6	Resolution de l'Exemple en obtenant son programme Pascal.	8
1.7	L'algorithme, une des étapes dans l'élaboration d'un Programme. . . .	10
1.8	Définition de Identifiant.	11
1.9	Exemple d'algorithme.	12
1.10	Définition de l'affectation, de l'incréméntation et de la Décrémentation. .	13
1.11	Opérateurs Booléens.	14
1.12	Opérateurs Booléens.	15
1.13	Opérateurs Réels.	15
1.14	Opérateurs Booléens.	16
1.15	Opérateurs Caracteres.	16
1.16	Opérateurs Booléens.	17
1.17	Abstraction de l'Univers Réel.	17
2.1	Exemple d'algorithme.	19
2.2	Resolution de l'Exemple 3.	19
2.3	Algorithme de l'Exemple 3.	20
2.4	Programme Pascal de l'Exemple 3.	20
2.5	Bloc d'Instructions.	20
2.6	Bloc d'Instructions.	21
2.7	Mon Premier Program.	21
2.8	Mon Deuxieme Program.	22
2.9	Exemple d'Action Alternative Simple.	22
2.10	Exemple d'Action Alternative Simple.	23
2.11	Exemple d'Action Alternative.	23
2.12	Action Alternative Complexe.	23

2.13	Action Alternative Multiple.	24
2.14	Exemple d'Action Itérative.	24
2.15	Exemple d'Action Itérative Tant Que.	24
2.16	Exemple d'Action Itérative Pour.	24
2.17	Exemple d'Action Itérative Répéter.	25

Chapitre 1

Généralités et Notions de Base

Summary

1.1	AVANT-PROPOS	2
1.2	Notion d'algorithme	4
1.2.1	Algorithme	4
1.2.2	Etapes de Résolution de Problèmes	4
1.2.3	Exemple	5
1.2.4	Exemple	6
1.2.5	Instance	6
1.2.6	Exemple de Résolution de Problème	6
1.2.7	Algorithme Correct	7
1.2.8	Exemple	7
1.2.9	Structure de Données	9
1.2.10	Applications	9
1.2.11	Efficacité d'un Algorithme	9
1.3	Notion de complexité	9
1.3.1	Complexité d'un Algorithme	9
1.3.2	Langage Algorithmique	10
1.4	Codage	11
1.4.1	Identifiant	11
1.4.2	Type Abstrait	11
1.4.3	Variable	12
1.4.4	Expression	12
1.4.5	Affectation	12
1.4.6	Types de base	12

1.1 AVANT-PROPOS

Ce polycopié pédagogique s'adresse aux étudiants de deuxième année du Département des Sciences Commerciales de la Faculté des Sciences économiques, de Gestion et Commerciales de l'Université Abderrahmane Mira de Béjaia. Il vise à rendre stimulant l'apprentissage des outils de base de l'informatique, à savoir :

- + les concepts de base du langage algorithmique et de l'organigramme, tels que :
 - * la notion de variable,
 - * de type de variable,
 - * affectation d'une valeur à une variable,
 - * expression,
 - * algorithme,
 - * instance,
 - * heuristique,
 - * efficacité d'un algorithme,
 - * notion de complexité,
 - * langage algorithmique et Organigramme,
 - * notion de codage et de structures de contrôle,
 - * et notion de structure de données (Tableaux, Matrices, ...),
- + Initiation à un Langage de Programmation Pédagogique à l'instar de Pascal
- + Quelques exemples d'algorithmes et de leur traduction en Pascal.

Le présent document couvre 14 séances de Travaux Pratiques, à raison d'une heure et demi par séance (01h30/ séance). L'ensemble des cours proposés se réfèrent au programme national des classes de Seconde Année des Sciences Commerciales.

Ce polycopié pédagogique intègre trois (3) compétences discursives suivantes :

- compréhension logique,
- programmation écrite et
- programmation sur machine.

Il est alors question de développer la compétence logique à travers l'écrit et la programmation écrite chez l'étudiant.

L'apprentissage de l'informatique répond à des exigences bien spécifiques, aussi bien à l'écrit, notamment : questions grammaticales, syntaxiques qu'à la Programmation (sur machine).

Le protocole suivi pour apprendre la programmation aux étudiants prévoit dans un premier temps que les travaux demandés soient relatifs au domaine des sciences économiques, et dans un second temps, que la première version soit corrigée par l'enseignant à travers une approche qui consiste à exploiter l'erreur en tant qu'outil d'apprentissage.

Cette pédagogie de l'erreur consiste d'abord à identifier les erreurs de programma-

tion (grammaticales, syntaxiques et lexicales) commises par l'étudiant. Il lui est demandé de les analyser, les corriger et les améliorer avec l'accompagnement de l'enseignant qui supervise.

L'expérimentation de cette méthode auprès des étudiants de deuxième année vise un apprentissage responsable via lequel l'étudiant apprend des astuces de programmation et acquiert un certain degré d'autonomie et un réflexe d'autocorrection et surtout de Programmation.

Il importe de préciser que l'ensemble des cours qui sollicitent et mobilisent les compétences informatiques de programmation sont dispensés sur 2 séances, et le reste des cours sur une séance.

En plus, des travaux sont répartis en groupes de 2 ou 3 étudiants à préparer chaque 15 jours ou carrément mensuels que ce soit à exposer ou à présenter à l'ensemble des étudiants (qui sera évalué).

Certaines activités requièrent une approche coopérative qui consiste et cette démarche amène l'étudiant à recenser ses lacunes et consolider ses acquis.

En d'autres termes, cette méthode l'aidera à discerner les points forts des points faibles et lui faire acquérir des compétences communicationnelles, telles que gérer son stress, parler en public et occuper de l'espace. à ce titre, une liste non exhaustive de sujets choisis par les apprenants pour l'épreuve orale pourrait révéler les orientations thématiques de ces derniers.

Plusieurs approches sont convoquées dans l'enseignement/apprentissage de l'économique à l'école, notamment les deux approches communicatives et motivationnelles qui sont essentiellement centrées sur l'apprenant. Elles ont pour principal but de créer un contexte qui favoriserait l'interaction et cultiverait la motivation chez des sujets ambitieux, et d'autres pas. En somme, l'approche motivationnelle se traduit par un système de récompense au lieu de sanction, de participation au lieu d'abstention et d'implication au lieu de marginalisation.

L'évaluation semestrielle des acquis est réalisée sous forme de contrôles continus d'examens de moyenne durée. à la fin de ce document pédagogique, s'ajoute une grille d'évaluation des enseignements qui a été élaborée par mes soins me permettant ainsi de faire un tour d'horizon, annuellement, sur ma pédagogie dans le but de l'améliorer, garantir un enseignement de qualité et un meilleur rendement.

Objectifs de l'enseignement

Cette matière permet aux étudiants :

- Logique de programmation
- conception d'organigramme
- familiarisation avec les systèmes de numérations de l'ordinateur

Connaissances préalables recommandées :

La maîtrise du français , des méthodes de calcul usuelles , des concepts de boules

Contenu de la matière

- 1- Résolution de problèmes à l'aide de l'ordinateur
- 2- Structure de la programmation
- 3- Les algorithmes
- 4- Les boucles

1.2 Notion d'algorithme

1.2.1 Algorithme

Un algorithme est une procédure de calcul bien définie qui prend en entrée un ensemble de valeurs et qui délivre en sortie un ensemble de valeurs.

Pour informatiser une application (faire réaliser par ordinateur, une tâche qui était réalisée par l'Homme), il faut :

- détailler suffisamment les étapes de résolution du problème, pour qu'elle soit exécutable par l'homme
- et transférer la résolution en une suite d'étapes si élémentaire et simple à exécuter (pouvant être codée en un programme dans un langage compréhensible par ordinateur).

Toute suite d'étapes si élémentaire et simple soit-elle à exécuter s'appelle un **ALGORITHME**.

Un Algorithme est le résultat d'une **démarche logique** (résolution du problème) en vue d'une mise en œuvre pratique sur ordinateur pour obtenir des **résultats concrets** (en passant par un langage de programmation).

Il consiste en une succession d'opérations, fidèlement exécutées, pour produire le résultat désiré.

C'est une suite d'actions élémentaires (instructions) à effectuer par un automate pour arriver à un résultat déterminé.

L'algorithmique est la logique d'écrire des algorithmes qui permet de :

- connaître la résolution manuelle du problème ;
- utiliser les capacités de l'ordinateur (actions élémentaires à assurer), et
- la logique d'exécution des instructions.

1.2.2 Etapes de Résolution de Problèmes

- 1 Comprendre l'énoncé du problème
- 2 Décomposer le problème en sous-problèmes plus simples à résoudre
- 3 Associer, à chaque sous-problème, une spécification :
 - des données nécessaires ;

- des données résultantes ;
- La démarche à suivre pour arriver au résultat en partant d'un ensemble de données

4. Et élaborer un algorithme.

Structure d'un algorithme

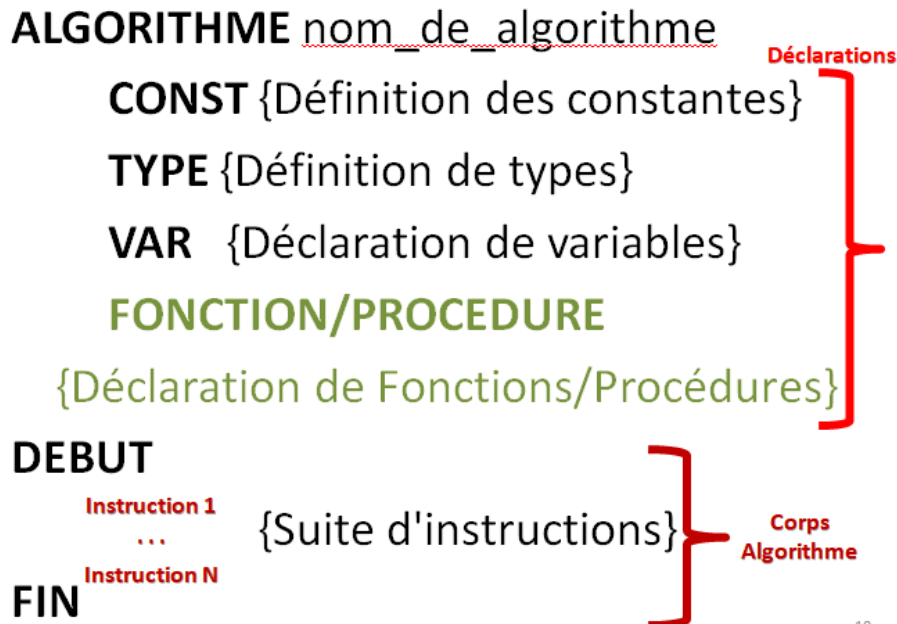


FIGURE 1.1 – Structure d'un algorithme.

1.2.3 Exemple

Exemple 1

```

ALGORITHME afficher
    { Aucune déclaration }
DEBUT
    Ecrire("La valeur de 3*5 est ", 3*5)
FIN
  
```

Cet algorithme permet d'afficher sur l'écran la phrase suivante :

La valeur de 3*5 est 15

FIGURE 1.2 – Exemple d'algorithme.

Cet algorithme permet d'afficher sur l'écran la phrase suivante :

La valeur de 3*5 est 15

1.2.4 Exemple

Trier une suite de nombres entiers dans l'ordre croissant.

Entrée :

Suite de n nombres entiers :

$$(a_1, a_2, \dots, a_n)$$

Sortie :

Une permutation de la suite donnée en entrée

$$(\check{a}_1, \check{a}_2, \dots, \check{a}_n)$$

telle que

$$\check{a}_1 \leq \check{a}_2 \leq \dots \leq \check{a}_n.$$

A partir de la suite

$$(6, 9, 2, 4),$$

un algorithme de tri fournira le résultat

$$(2, 4, 6, 9).$$

1.2.5 Instance

Une valeur particulière de l'ensemble des valeurs données en entrée est appelée **Instance du problème**.

Exemple

La valeur

$$(6, 9, 2, 4)$$

est une instance du problème.

1.2.6 Exemple de Résolution de Problème

On veut écrire l'algorithme qui permet de lire 2 valeurs et de calculer la moyenne :

Pour cela, il faut d'abord Rechercher la solution en 2 phases, à savoir :

- Comprendre comment résoudre manuellement,
- Définir données, formules de calcul et résultats
- Problème : $(A + B) / 2$

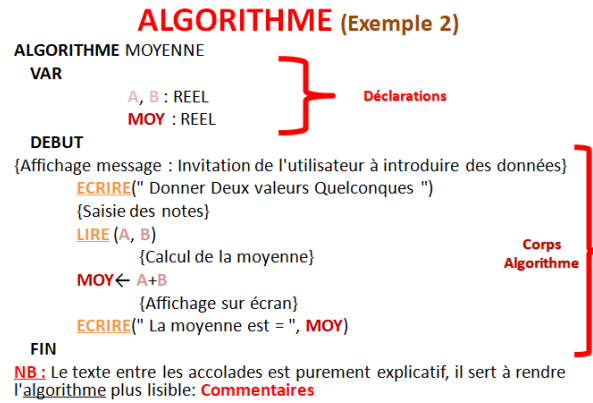


FIGURE 1.3 – Exemple d'algorithme.

1.2.7 Algorithme Correct

Un algorithme est correct si pour toute instance du problème il se termine et produit une sortie correcte.

Les algorithmes peuvent être spécifiés en langage humain ou tout langage informatique.

1.2.8 Exemple

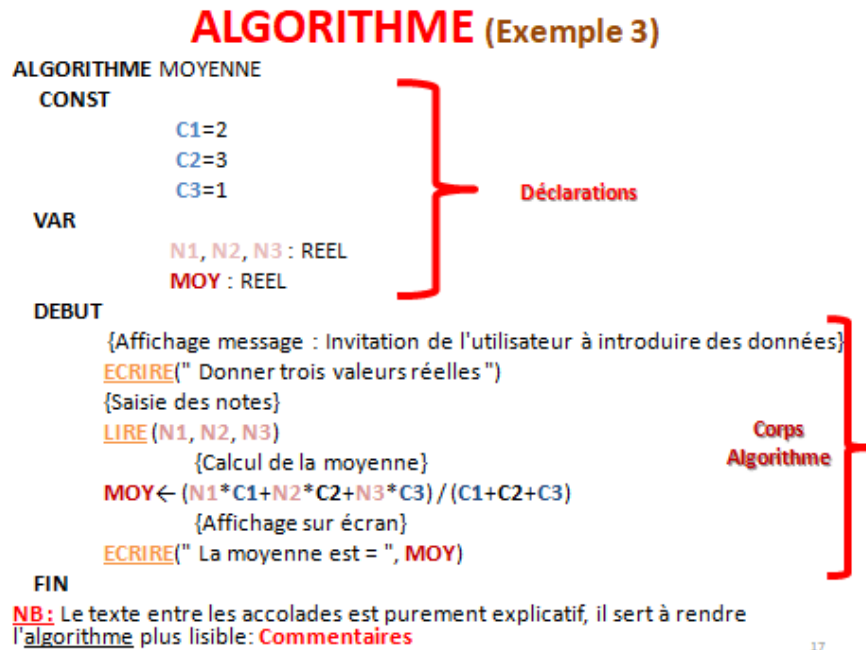
On veut écrire l'algorithme qui permet de saisir 3 notes d'un étudiant dans trois matières, avec les coefficients respectifs 2, 3 et 1 et de calculer la moyenne.

Résolution

- ➔ Recherchons la solution en 2 phases
- ➔ Comprendre comment résoudre manuellement,
- ➔ Définir **données, formules de calcul et résultats**
- ➔ Problème: $\sum N_i * C_i / \sum C_i$

16

FIGURE 1.4 – Resolution de l'Exemple 3.



17

FIGURE 1.5 – Resolution de l'Exemple en obtenant un Algorithme.

Programme Pascal (Exemple 3)

```

program calcul_moy;
uses crt;
var c1,c2,c3,n1,n2,n3,moy:real;

begin
  clrscr;

  write('donner le coef c1 svp : ');
  readln(c1);
  write('donner le coef c2 svp : ');
  readln(c2);
  write('donner le coef c3 svp : ');
  readln(c3);

  write('donner la note de n1 svp : ');
  readln(n1);
  write('donner la note de n2 svp : ');
  readln(n2);
  write('donner la note de n3 svp : ');
  readln(n3);

  moy:= (c1*n1 + c2*n2 + c3*n3)/ (c1 + c2 + c3);

  writeln('la moyenne est =',moy);

  readln;
end.
  
```

19

FIGURE 1.6 – Resolution de l'Exemple en obtenant son programme Pascal.

Dans ce qui suit nous utiliserons un langage proche du langage naturel.
Nous donnerons une implémentation en Pascal.

1.2.9 Structure de Données

Une structure de données est un moyen de stocker et d'organiser des données pour faciliter leur stockage, leur utilisation et leur modification.

1.2.10 Applications

De nombreux problèmes nécessitent des algorithmes :

- Bioinformatique
- Moteur de recherche sur Internet
- Commerce électronique
- Affectation de tâches
- ...

1.2.11 Efficacité d'un Algorithme

L'efficacité d'un algorithme est mesurée par son coût (complexité) en temps et en mémoire.

1.3 Notion de complexité

L'efficacité d'un algorithme est fondamentale pour résoudre effectivement des problèmes.

Exemple

Supposons que l'on dispose de deux ordinateurs. L'ordinateur A est capable d'effectuer 109 instructions par seconde. L'ordinateur B est capable d'effectuer 107 instructions par seconde.

Considérons un même problème (de tri par exemple) dont la taille des données d'entrées est n . Pour l'ordinateur A, on utilise un algorithme qui réalise $2n^2$ instructions. Pour l'ordinateur B, on utilise un algorithme qui réalise $50n\log(n)$ instructions.

Pour traiter une entrée de taille 106 : l'ordinateur A prendra 2000s et l'ordinateur B prendra 100s.

Ainsi, même si la machine B est médiocre, elle résoudra le problème 20 fois plus vite que l'ordinateur A.

1.3.1 Complexité d'un Algorithme

La complexité d'un algorithme est en temps, le nombre d'opérations élémentaires effectuées pour traiter une donnée de taille n , en mémoire, l'espace mémoire néces-

saire pour traiter une donnée de taille n .

Dans ce cours, nous considérerons que la complexité des instructions élémentaires les plus courantes sur un ordinateur ont un temps d'exécution que l'on considérera dans ce cours comme constant égal à 1.

Les instructions élémentaires sont : addition, multiplication, modulo et partie entière, affectation, instruction de contrôle. Ce qui intéresse fondamentalement l'algorithmique c'est l'ordre de grandeur (au voisinage de l'infini) de la fonction qui exprime le nombre d'instructions.

Un programme est un algorithme codé (traduit) dans un langage compréhensible par l'ordinateur à l'aide d'un compilateur (traducteur) (Pascal, ...)

1.3.2 Langage Algorithmique

Il est nécessaire de disposer d'un langage qui soit non lié à l'implémentation. Ceci permet une description plus précise des structures de données ainsi qu'une rédaction de l'algorithme plus souple et plus **lisible**. Le langage algorithmique est un exemple de ce

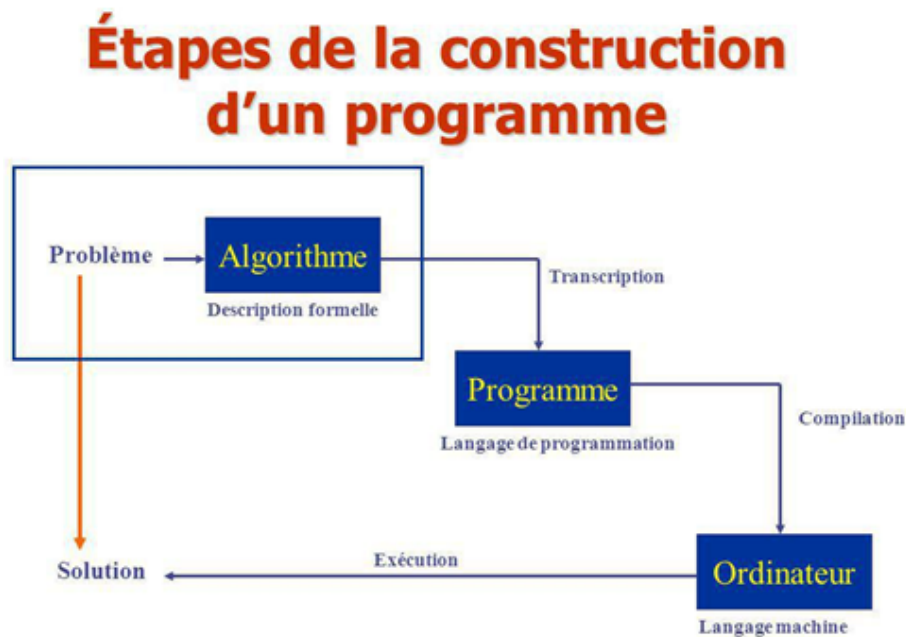


FIGURE 1.7 – L'algorithme, une des étapes dans l'élaboration d'un Programme.

qui peut être utilisé et qui sera utilisé dans ce cours. Il est composé de chaînes de caractères alphanumériques, de signes opératoires (+, -, *, /, <, <=, >=, >, <>, ==, !=, ou, non, et), de mot-clés réservés, et de signes de ponctuation : "=", ";", "(", ")", "début", "fin", "//".

Remarque

Pascal aussi utilise des marqueurs de début (Begin) et de fin (End).

1.4 Codage

1.4.1 Identifiant

Identifiant

Nom, donné par l'utilisateur, de:

- ➔ Constante,
- ➔ Variable,
- ➔ Algorithme (Programme),
- ➔ Fonction
- ➔ Ou Procédure

Qui est une suite de caractères alphanumériques dont le premier caractère est, **obligatoirement, alphabétique**, peut accepter '_' (souligné) au milieu

Exemple: A_1, Nom_User_1, Nom sont des identifiants Mais 3A, _Nom ne sont acceptés comme identifiants

NB: Surtout Pas de caractères spéciaux : !, ?, *, =, ... 45

FIGURE 1.8 – Définition de Identifiant.

1.4.2 Type Abstrait

Un **type abstrait** est un triplet composé :

- d'un nom,
- d'un ensemble de valeurs,
- d'un ensemble d'opérations définies sur ces valeurs.

Les types abstraits de base de l'algorithmique sont :

- entier,
- caractères,
- booléen,
- réel

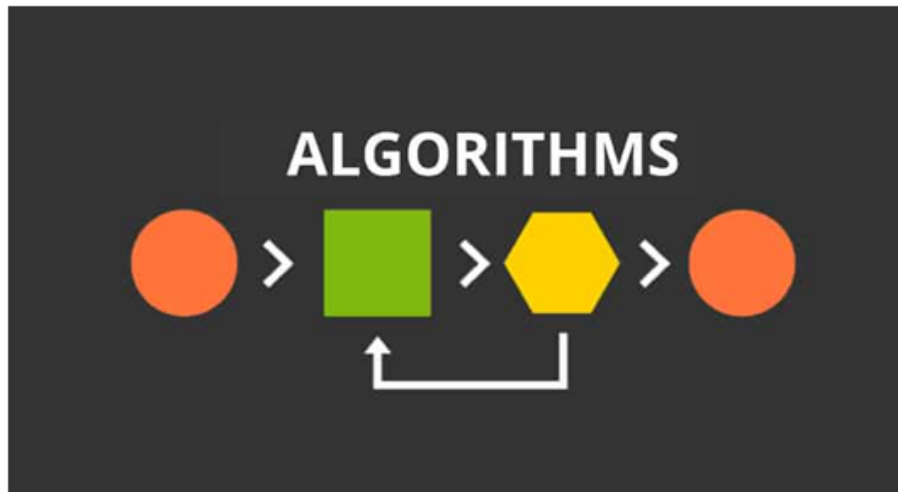


FIGURE 1.9 – Exemple d'algorithme.

1.4.3 Variable

Une variable est un triplet composé d'un type (déjà défini), d'un identifiant et d'une valeur.

var*NomDeVariable* : **Type**;

Type est à prendre pour l'instant dans l'ensemble : {entier, car, booléen, réel}

1.4.4 Expression

Les Expressions sont constituées à l'aide de variables déjà déclarées, de valeurs, de parenthèses et d'opérateurs du (des) type(s) des variables concernées.

1.4.5 Affectation

L'affectation est l'instruction qui permet de stocker une valeur dans une variable.
On écrit

NomDeVariable := **ExressionDuTypeDeLaVariable**;

Remarque

Toute variable doit être déclarée et recevoir une valeur initiale.

1.4.6 Types de base

Les types de base sont :

Booléens Une variable de type booléen prend comme valeur VRAI ou FAUX.

Affectation

Une **variable** est modifiée dans un programme à l'aide de **l'affectation** ($\leftarrow$):

Exemple:

$\text{Moy} \leftarrow 0;$

Ou d'une **Incrémentation**:

$I \leftarrow I + 1;$

Ou d'une **Décrémentation**:

$N \leftarrow N - 1;$

57

FIGURE 1.10 – Définition de l'affectation, de l'incréméntation et de la Décrémentation.

Les opérations usuelles sont **ET**, **OU** et **NON** qui sont données dans les tables qui suivent.

TABLE 1.1 – Table Logique **ET**

ET	Faux	Vrai
Faux	Faux	Faux
Vrai	Faux	Vrai

TABLE 1.2 – Table Logique **OU**

OU	Faux	Vrai
Faux	Faux	Vrai
Vrai	Vrai	Vrai

Entiers Une variable de type entier peut prendre comme valeur l'ensemble des nombres entiers signés. Les opérations associées sont les opérations usuelles $+$, $-$, $*$, $/$.

Réels Une variable de type réel peut prendre comme valeur l'ensemble des nombres réels. Les opérations associées sont les opérations usuelles $+$, $-$, $*$, $/$.

Caractères Une variable de type car peut prendre comme valeur l'ensemble des caractères imprimables. On notera les valeurs entre guillemets. On considère souvent que les caractères sont ordonnés dans l'ordre alphabétique.

Attention

TABLE 1.3 – Table Logique NON

NON	
Faux	Vrai
Vrai	Faux

Opérateurs

Booléens:

Opérations Possibles	Symbole ou Mot-Clé	Résultat
Comparaisons	=, ≠	Booléen
Négation	NON	Booléen
Conjonction	ET	Booléen
Disjonction	OU	Booléen

63

FIGURE 1.11 – Opérateurs Booléens.

Les valeurs

- o "1" qui est un caractère,
- o 1 qui est un entier,
- o 1. qui est un réel

sont différentes et ne seront pas codés de la même manière dans la mémoire de la machine.

1.4.6.1 Comparaison

Les opérateurs $<$, $\leq$, $=$, $\neq$, $>$, $\geq$ permettent de comparer les valeurs de type entier, réel et caractère. Le résultat de cette comparaison est une valeur booléenne.

Priorités des Opérateurs

Booléens:



Priorités	Symbole ou Mot-Clé
	ET OU
	NON
	=, ≠

65

FIGURE 1.12 – Opérateurs Booléens.

Opérateurs

Réels:

Opérations Possibles	Symbole ou Mot-Clé	Résultat
Addition	+	Réel
Soustraction	-	Réel
Multiplication	•	Réel
Division	/	Réel
Comparaisons	<, ≤, >, ≥, =, ≠	Booléen

57

FIGURE 1.13 – Opérateurs Réels.

Priorités des Opérateurs

Réels:

-



+

Priorités	Symbole ou Mot-Clé
Addition	- +
Soustraction	* /
Multiplication	<, ≤, >, ≥, =, ≠

59

FIGURE 1.14 – Opérateurs Booléens.

Opérateurs

Caractères:

Opérations Possibles	Symbole ou Mot-Clé	Résultat
Comparaisons	<, ≤, >, ≥, =, ≠	Booléen
	Exemples: 'a' < 'z'; 'a' ≠ 'b'	Booléen

61

FIGURE 1.15 – Opérateurs Caracteres.

Priorités des Opérateurs

Entiers:

-
 ↓
 +

Priorités	Symbole ou Mot-Clé
0	+ -
1	* /
2	DIV MOD
3	<, ≤, >, ≥, =, ≠

55

FIGURE 1.16 – Opérateurs Booléens.



FIGURE 1.17 – Abstraction de l'Univers Réel.

Chapitre 2

Structures de Contrôles

Summary

2.1	Structures de Contrôle	18
2.2	Bloc d'instruction	18
2.2.1	Action Conditionnelle	18
2.2.2	Action Itérative	18

2.1 Structures de Contrôle

Il y a trois structures principale de contrôle qui permettent de construire des algorithmes

2.2 Bloc d'instruction

2.2.1 Action Conditionnelle

2.2.1.1 Alternative

2.2.1.2 Alternative simple :

2.2.1.3 Alternative multiple :

2.2.2 Action Itérative

- Répétition

L'instruction **exit** permet d'arrêter la répétition.

- o le bloc d'instruction peut ne pas être exécuté :

- o le bloc d'instruction peut ne pas être exécuté et il y a une variable indicatrice :

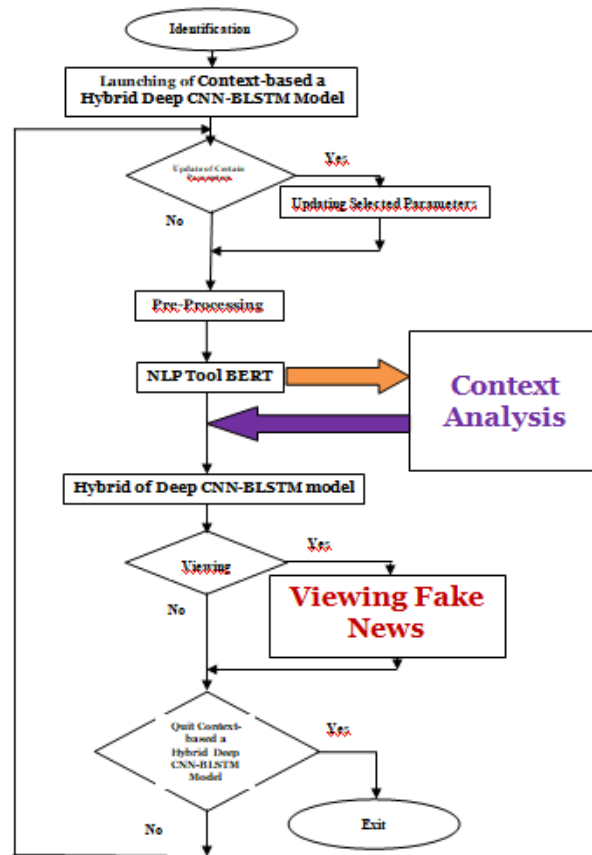


FIGURE 2.1 – Exemple d'algorithme.

Résolution

- ➔ Recherchons la solution en 2 phases
- ➔ Comprendre comment résoudre manuellement,
- ➔ Définir **données, formules de calcul** et **résultats**
- ➔ Problème: $\sum N_i * C_i / \sum C_i$

16

FIGURE 2.2 – Resolution de l'Exemple 3.

o le bloc d'instruction est exécuté au moins une fois :

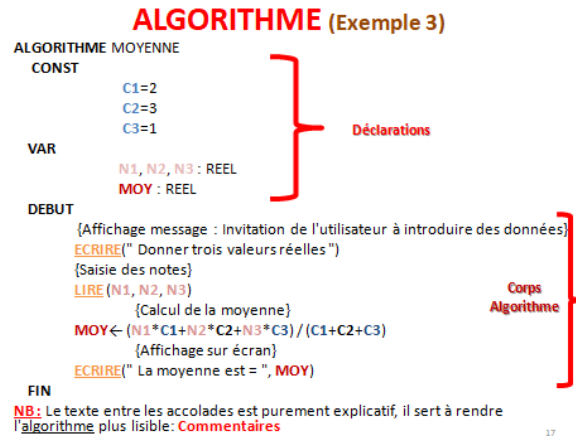


FIGURE 2.3 – Algorithme de l'Exemple 3.

Programme Pascal (Exemple 3)

```

program calcul_moy;
uses crt;
var c1,c2,c3,n1,n2,n3,moy:real;

begin
  clrscr;

  write('donner le coef c1 svp : ');
  readln(c1);
  write('donner le coef c2 svp : ');
  readln(c2);
  write('donner le coef c3 svp : ');
  readln(c3);

  write('donner la note de n1 svp : ');
  readln(n1);
  write('donner la note de n2 svp : ');
  readln(n2);
  write('donner la note de n3 svp : ');
  readln(n3);

  moy:= (c1*n1 + c2*n2 + c3*n3)/(c1 + c2 + c3);

  writeln('la moyenne est =',moy);

  readln;
end.
  
```

19

FIGURE 2.4 – Programme Pascal de l'Exemple 3.

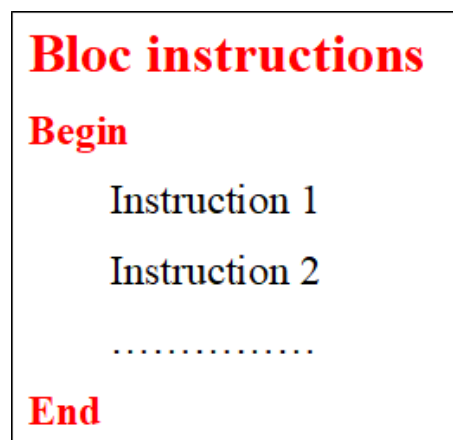


FIGURE 2.5 – Bloc d'Instructions.

```
Program MonProg;  
  
var  
  
    i : integer;  
  
    j : string;  
  
begin  
  
    i:=1;  
  
    j:='Bonjour à tous';  
  
end.
```

FIGURE 2.6 – Bloc d'Instructions.

```
Program MonPremierProg  
  
var  
  
    s : string;  
  
begin  
  
    s := 'Salut à tous !';  
  
    writeln(s);  
  
    readln;  
  
end.
```

FIGURE 2.7 – Mon Premier Program.

```
Program MonDeuxiemeProg  
var  
    s, Phrase : string;  
begin  
    writeln('Entrez un mot :');  
    readln(s);  
    Phrase:=s;  
    writeln('Entrez un deuxième mot :');  
    readln(s);  
    Phrase:=Phrase+s ;  
    writeln('Entrez un troisième mot :');  
    readln(s);  
    Phrase:=Phrase+s;  
    writeln('La phrase complète est :');  
    writeln(Phrase);  
end.
```

FIGURE 2.8 – Mon Deuxieme Program.

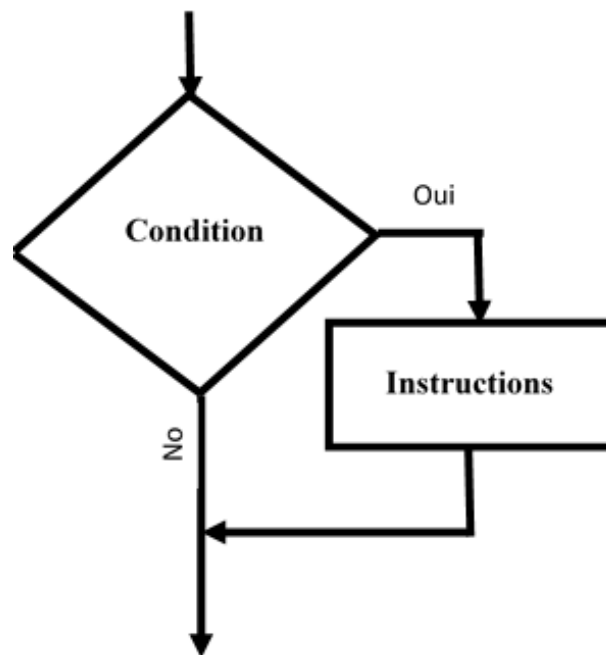


FIGURE 2.9 – Exemple d'Action Alternative Simple.

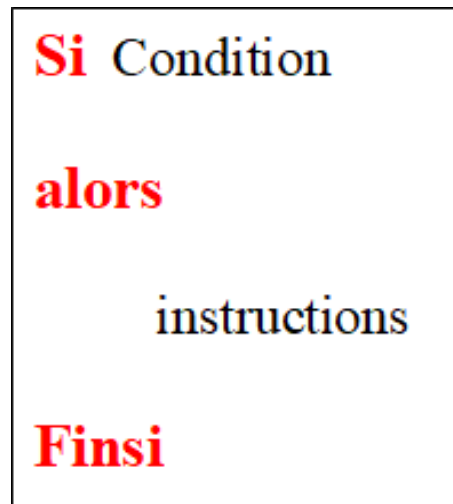


FIGURE 2.10 – Exemple d'Action Alternative Simple.

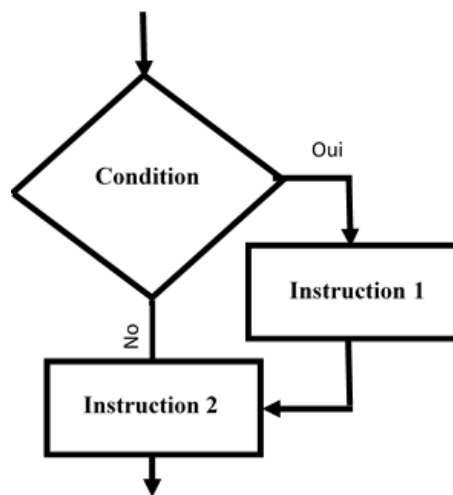


FIGURE 2.11 – Exemple d'Action Alternative.

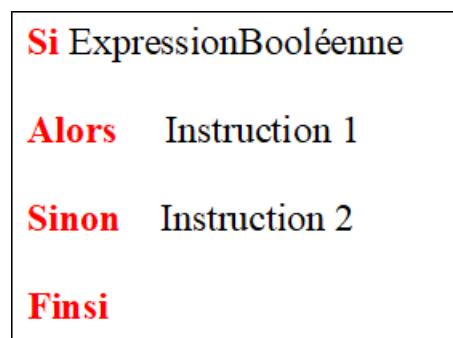


FIGURE 2.12 – Action Alternative Complexe.

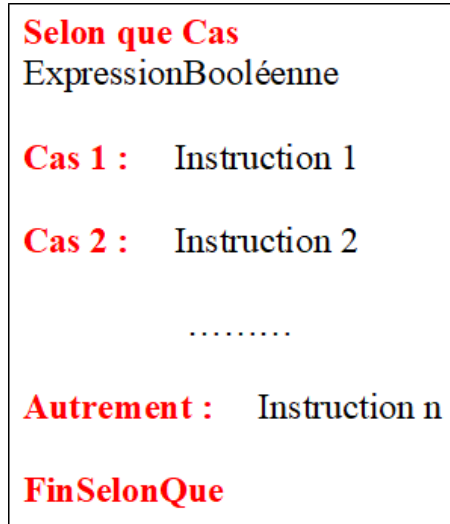


FIGURE 2.13 – Action Alternative Multiple.

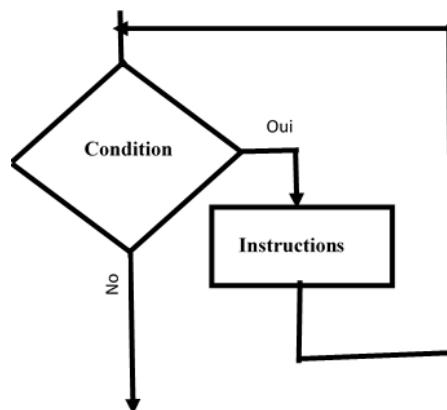


FIGURE 2.14 – Exemple d'Action Itérative.

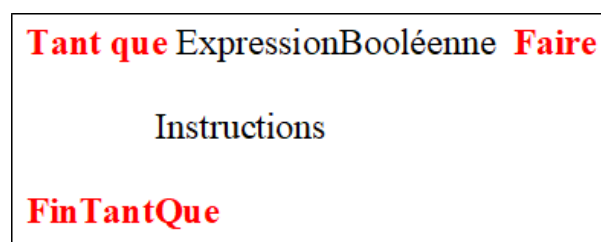


FIGURE 2.15 – Exemple d'Action Itérative Tant Que.

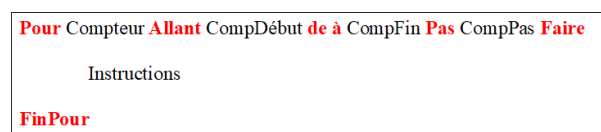


FIGURE 2.16 – Exemple d'Action Itérative Pour.

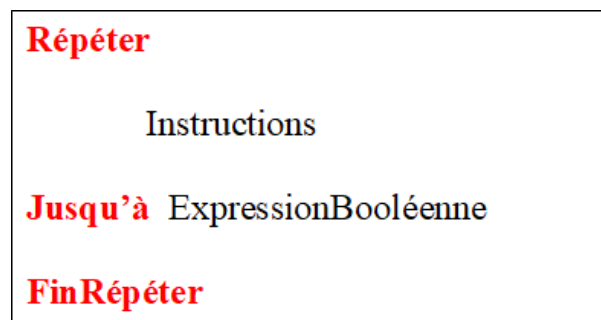


FIGURE 2.17 – Exemple d'Action Itérative Répéter.

Chapitre 3

Structures de Données

Summary

3.1	Tableaux	26
3.1.1	Quelques exemples d'algorithmes	27
3.2	Matrices	28
3.2.1	Initialisation	29
3.2.2	Somme de deux matrices réelles	29

3.1 Tableaux

Un tableau est une table d'association à clé unique telle que

- o le nombre d'éléments de la table (dimension ou taille) est constant,
- o l'accès aux éléments s'effectue directement par la clé,
- o les valeurs minimum et maximum des clés sont des constantes.

On écrit en algorithmique

$nom_tableau = \mathbf{tableau}[min_indice..max_indice]detype_predefini;$

ce qui signifie que

- les éléments ont pour type le `type_predefini`
- les indices des éléments vont de `min_indice` à `max_indice`, avec :

$$min_indice < max_indice$$

La taille du tableau est donc :

$$max_indice - min_indice + 1.$$

Pour accéder à un élément d'un tableau T d'indice I , on écrit

$$T[I].$$

La complexité de l'accès à un élément du tableau est

$$O(1).$$

3.1.1 Quelques exemples d'algorithmes

3.1.1.1 Somme des éléments d'un tableau d'entiers

fonction*somme*(**ref** T : **tableau** $[min_indice..max_indice]$ d'entiers) : entier;

var s, i : entier ;

début

$s=0$;

pour i allant de min_indice à max_indice **faire**

$s=s+T[i]$

finpour

retourner(s)

fin

Complexité : $O(n)$

3.1.1.2 Recherche d'un élément

Propriété

Soit i, j deux entiers, $i \leq j$.

Soit T un tableau d'éléments d'indice variant entre i et j . Pour tout élément e , appartenant au tableau T , on a $T[i]=e$ ou e est dans $T[i+1..j]$

fonction*cherche*(**ref** T : **tableau** $[min_indice..max_indice]$ d'lment; **vale** : lment) : entier;

var

i : entier ;

début

pour i allant de min_indice à max_indice **faire**

si $T[i]==e$ alors

```
retourner(i)
```

```
finpour
```

```
retourner()
```

```
fin
```

```
finfonction
```

Complexité minimum : $O(1)$ maximum : $O(n)$ Source Python

3.1.1.3 Recherche de l'indice du premier élément minimum

On suppose que le tableau contient des éléments comparables (l'ensemble des éléments est muni d'une relation d'ordre).

Choisissons ici, pour simplifier les notations, des entiers.

Propriété

Soit i, j deux entiers, $i \leq j$.

Soit T un tableau d'entiers d'indice variant entre i et j .

Soit m l'élément minimum du tableau, on a $T[i]=m$ ou m est dans $T[i+1..j]$

```
fonction minimum(ref $T$  : tableau[min_indice..max_indice]d'entier) : entier;
```

```
var
```

```
 $i$ ,sauv :entier ;
```

```
début
```

```
sauv= min_indice ;
```

```
pour  $i$  allant de min_indice+1 à max_indice faire
```

```
si  $T[i]$ 
```

3.2 Matrices

Les Déclaration

Une matrice M de dimension $n \times m$ est un tableau de dimension n dont chaque élément est un tableau de dimension m .

On peut donc déclarer la matrice sous la forme suivante `var M :tableau[1..n]de tableau [1..m] d'éléments ;`

3.2.1 Initialisation

```

fonction initMatrice(ref M : tableau[1..n]de tableau[1..m]d'lments; val valeurInitiale : lment) :

    var i,j :entier ;
    début
    pour i allant de 1 à n faire
    pour j allant de 1 à m faire
        M[i][j]=valeurInitiale
    finpour
    finpour
    retourner()
finfonction
Complexité :  $O(nm)$ 

```

3.2.2 Somme de deux matrices réelles

^{es}
L

```

fonction sommeMatrice(ref M1, M2 : tableau[1..n]de tableau[1..m]de réels) : tableau[1..n]de réels

    var i,j :entier ;
    var M :tableau[1..n] de tableau [1..m] de réels ;
    début
    pour i allant de 1 à n faire
    pour j allant de 1 à m faire
        M[i][j]=M1[i][j]+M2[i][j] ;
    finpour
    finpour
    retourner(M)
finfonction
Complexité :  $O(nm)$ 

```

Conclusion

Cette section clôt ce polycopé d'initiation à l'algorithmique et à la programmation. Tout au long de ce polycopé, nous avons présenté les trois chapitres qui le composent.

Le chapitre 1 est constitué des concepts de données, de variables, des différents types de variables, d'initialisation, d'affectation, d'incrément, de décrémentation d'une valeur ou d'une expression à une variable.

Ensuite, nous sommes passé à l'évaluation des différents types d'expressions en respectant l'ordre de priorité des opérateurs.

Tout cela est suivi par les opérations d'entrée / sortie.

Le chapitre 2 représente les structures de contrôle de l'ensemble de contrôle.

Le chapitre 3 constitue l'ensemble des structures de données telles que tableau à une dimension (Tableau) et tableau à deux dimensions (matrice).

Références

- [1]
- [2] 2- Rifflet Jean-Marie, Yunès Jean-Baptiste : Fondements de la programmation (concepts et techniques) édition ELLIPSES 2014.
- [3] 3- Auteurs collectif : Initiation à l'informatique . édition Ini. 2013
- [4] Bouzidi Z., Boudries A. and Amad M., (2018), *A New Efficient Alert Model for Disaster Management*. In *Proceedings of Conference AIAP'2018 : Artificial Intelligence and Its Applications, El-Oued, Algeria*
- [5] Bouzidi, Z., Amad, M. and Boudries, A., (2019), *Intelligent and Real-time Alert Model for Disaster Management based on Information retrieval from Multiple Sources*. International Journal of Advanced Media and Communication, Vol. 7, No. 4
- [6] Bouzidi, Z., Boudries, A. and Amad, M., (2020), *Towards a Smart Interface-based Automated Learning Environment Through Social Media for Disaster Management and Smart Disaster Education*. Arai K., Kapoor S., Bhatia R. (eds) Intelligent Computing. (SAI 2020), Advances in Intelligent Systems and Computing, vol. 1228, pp. 443-468, Springer, Cham., [https : //doi.org/10.1007/978 – 3 – 030 – 52249 – 0_31](https://doi.org/10.1007/978-3-030-52249-0_31)
- [7] Bouzidi, Z., Boudries, A. and Amad, M. (2021), *Deep learning-based Model to improve Warning, Situational Awareness and Education in Managing Emergency : Case study of Covid-19*. Ingénierie des Systèmes Information, Vol., No., Unpublished
- [8] Bouzidi, Z., Boudries, A. & Amad, M., (2021), *Deep Convolutional Neural Network-LSTM-based Model to improve Warning, Situational Awareness and Education in Managing Emergency*. International Journal of Safety and Security Engineering, Vol., No., Month, Year, pp. 1-8, Journal homepage : [http ://iieta.org/journals/ijssse](http://iieta.org/journals/ijssse)
- [9] Bouzidi, Z., Amad, M. and Boudries, A. (2020b), *Deep Learning-based Automated Learning Environment using Smart Data to improve Corporate Marketing, Business Strategies, Fraud Detection in Financial Services and Financial Time*

- Series Forecasting*. In International Conference on Managing Business through Web Analytics - (ICMBWA2020), Khemis Miliana University, Algeria, Accepted
- [10] Bouzidi, Z., Boudries, A. and Amad, Mourad, (2021c), *Enhancing Crisis Management because of Deep Learning, Big Data and Parallel Computing Environment : Survey*. ID : 443, Proceedings of the 3rd International Conference on Electrical, Communication and Computer Engineering (ICECCE), 12–13 June 2021, Kuala Lumpur, Malaysia, pp. 1-7, doi : 10.1109/ICECCE52056.2021.9514189
- [11] Bouzidi, Z., Boudries, A. and Amad, Mourad, (2021b), *LSTM-based automated learning with smart data to improve marketing fraud detection and financial forecasting*. 5th International Scientific Conference on Economics and Management, (EMAN 2021), Serbia, Accepted
- [12] Boudries, A., Amad, M. and Siarry, P., (2016), *Novel approach for replacement of a failure node in wireless sensor network*. Journal of Telecommunication Systems, DOI : 10.1007/s1123501602365
- [13] Bouzidi, Z., Amad, M. and Boudries, A. (2021d), *A Survey on Deep Learning in Big Data and its Applications*. International Conference on Innovations in Energy Engineering & Cleaner Production (IEECP 21), ID : 124, Silicon Valey, California, USA, DOI :<https://dx.doi.org/10.6084/m9.figshare.14737953>